

Real-Time Video Ingestion & Human Activity Understanding

A 40-Hour Prototype Implementation Plan

Target: PS-TR-1, Round 1 (Questions 1.1, 1.2, 1.3, 2.1, 2.2, 2.3) plus the demo round

Hardware assumption: Fedora laptop (4 GB RAM, 512 GB HDD, no GPU) plus one free-tier ARM cloud VM

Guiding principle: You are not building a surveillance product. You are building an *instrument that measures* a surveillance product. Every hour that does not produce a measurement, a failure, or a verdict scores nothing.

How to use this document

Each phase has an objective, a time budget, numbered subtasks, exact commands, a “done when” gate, and a list of failure modes that will actually bite you.

Do not skip the gates. If a phase’s gate does not pass, the next phase will silently produce garbage data, and you will discover it at hour 35 when there is no time to fix it.

Three rules that override everything else in this document:

- Write `common/telem.py` in hour one.** Every measurement is free if the log schema exists from the start and unrecoverable if it does not.
- Timebox Oracle Cloud provisioning to two hours.** It is the highest-variance, least interesting problem in the plan. Fall back to a paid €4 VPS rather than lose a week.
- Never cut the log schema, the outage run, the adjudicator, or the measurement pass.** Those four are the entire submission.

Time budget at a glance

Phase	Content	Hours	Cumulative
0	Environment and accounts	4	4
1	Camera simulation and datasets	3	7
2	Telemetry schema and edge agent	8	15
3	Inference cascade	7	22
4	Cloud plane and adjudicator	6	28
5	Observability	3	31
6	Chaos runs and measurement	5	36
7	Two HTML pages	2	38
8	Demo recording and written answers	2	40

Architecture recap

One cloud VM hosts **two logical sites**, not two VMs. Site A runs ShanghaiTech scenes 01 to 06, Site B runs scenes 07 to 13, as separate `edge-agent` processes with separate outboxes, separate SQLite manifests, and separate `site_id` values.

This gives you genuine location cohorts for Question 2.2 at a fraction of the setup cost. Failure injection runs through Toxiproxy between edge and cloud, so you can throttle one site independently without touching network namespaces.

State this openly in your write-up: sites are logically separated on shared hardware. Scoped honesty scores well. Pretending does not.

Laptop (Fedora) services	Cloud VM (ARM)	Managed
-----	-----	-----
browser + ssh (Postgres)	mediamtx (12 RTSP streams)	Neon
2-camera local test	edge-agent-A (scenes 01-06)	Claude /
Gemini API	edge-agent-B (scenes 07-13)	
code editor	toxiproxy	
	FastAPI cloud plane	
	adjudicator worker	
	prometheus + grafana	

Phase 0 — Environment and Accounts

Objective: Every tool installed, every account live, SSH working, database reachable from the VM.

Budget: 4 hours. **Do not exceed 5.** If you are over, drop Oracle and buy a €4 Hetzner box.

0.1 — Fedora laptop base packages (20 min)

Enable RPM Fusion first, because Fedora's stock repos ship a crippled ffmpeg without the codecs you need.

```
sudo dnf install -y \
  https://mirrors.rpmfusion.org/free/fedora/rpmfusion-free-release-
$(rpm -E %fedora).noarch.rpm \
  https://mirrors.rpmfusion.org/nonfree/fedora/rpmfusion-nonfree-
release-$(rpm -E %fedora).noarch.rpm

sudo dnf install -y ffmpeg ffmpeg-devel iproute-tc jq git podman \
  python3-pip python3-devel gcc sqlite openssl-
clients \
  wget curl unzip
```

Verify the codec set is real, not the stub:

```
ffmpeg -codecs 2>/dev/null | grep -E "h264|hevc"
```

You must see `libx264` in the encoders column. If you only see decoders, RPM Fusion did not install and you need to re-run the first command.

0.2 — Python environment (10 min)

Never install into system Python on Fedora. It breaks `dnf`.

```
python3 -m venv ~/.vi-env
source ~/.vi-env/bin/activate
echo 'source ~/.vi-env/bin/activate' >> ~/.bashrc

pip install --upgrade pip
pip install av onnxruntime opencv-python-headless requests \
  fastapi uvicorn psycpg2-binary prometheus-client \
  pyyaml pillow numpy
```

Use `opencv-python-headless`, not `opencv-python`. The full package pulls in Qt and roughly 400 MB of GUI dependencies you will never use. On a 4 GB machine that matters.

You will use OpenCV for exactly one thing: the motion gate. Stream capture goes through PyAV.

0.3 — Project skeleton (10 min)

```
mkdir -p ~/video-
intel/{common,edge,cloud,sim,ops,bench,web,docs,logs}
cd ~/video-intel
git init
```

Create `.gitignore`:

```
cat > .gitignore <<'EOF'
__pycache__/
*.pyc
logs/*.jsonl
data/
*.onnx
*.db
.env
venv/
EOF
```

Create `.env` (never commit it):

```
cat > .env <<'EOF'
DATABASE_URL=postgresql://user:pass@host/db
ANTHROPIC_API_KEY=sk-ant-xxxx
CLOUD_API=http://127.0.0.1:8000
EOF
```

0.4 — Oracle Cloud VM (90 min, hard timebox)

1. Sign up at `cloud.oracle.com`. A card is required for identity verification; Always Free resources do not charge it.
2. Choose your home region carefully. **This cannot be changed later.** Mumbai and Hyderabad are frequently at capacity for ARM. Singapore and Frankfurt are usually available and latency from India is acceptable (roughly 50 to 100 ms, irrelevant for this workload).
3. Compute → Instances → Create Instance.
4. Image: Ubuntu 22.04 or Oracle Linux 9. Either is fine; commands

below assume Ubuntu.

5. Shape: change from the default → Ampere → **VM.Standard.A1.Flex**. Request 2 OCPU and 12 GB RAM. If the create button errors with “Out of capacity,” retry every 15 minutes or switch region.
6. Add your SSH public key (`~/.ssh/id_ed25519.pub` ; generate with `ssh-keygen -t ed25519` if you have none).
7. Create a 100 GB block volume, attach it, and mount it at `/data`.

Open the required ports. Two layers must both allow traffic, and forgetting the second is the most common Oracle mistake.

Layer 1, the OCI security list (Networking → VCN → Security Lists → Default → Add Ingress Rules): allow TCP 22, 8000, 3000, 8554, 9090 from `0.0.0.0/0`.

Layer 2, the instance firewall (Ubuntu images ship with restrictive iptables rules preinstalled by Oracle):

```
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 8000 -j ACCEPT
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 3000 -j ACCEPT
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 8554 -j ACCEPT
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 9090 -j ACCEPT
sudo netfilter-persistent save
```

Fallback if Oracle refuses after 2 hours: provision a Hetzner CX22 (2 vCPU, 4 GB, €3.79/month). Everything in this document works identically on it, with fewer cameras. Roughly ₹400 is not worth a week of retry loops.

0.5 — VM base setup (20 min)

```
ssh ubuntu@<VM_IP>

sudo apt update && sudo apt install -y \
  ffmpeg python3-pip python3-venv sqlite3 jq podman \
  git wget curl unzip build-essential

# Mount the block volume
sudo mkfs.ext4 /dev/oracleoci/oraclelvdb
sudo mkdir -p /data
sudo mount /dev/oracleoci/oraclelvdb /data
sudo chown -R ubuntu:ubuntu /data
echo '/dev/oracleoci/oraclelvdb /data ext4 defaults,_netdev,nofail 0 2' | sudo tee -a /etc/fstab

python3 -m venv ~/vi-env
source ~/vi-env/bin/activate
pip install av onnxruntime opencv-python-headless requests fastapi \
  uvicorn psycpg2-binary prometheus-client pyyaml pillow
numpy
```

ARM note: `onnxruntime` has prebuilt aarch64 wheels and installs cleanly. If pip tries to build from source, you are on an unsupported Python version; use 3.10 or 3.11.

Add swap, because 12 GB is comfortable but ONNX model loading spikes:

```
sudo fallocate -l 4G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

0.6 — Neon Postgres (15 min)

1. Sign up at neon.tech. Free tier, no card.
2. Create a project named `video-intel`, region Singapore or Frankfurt to match your VM.
3. Copy the connection string into `.env` on both machines.
4. Verify from the VM:

```
psql "$DATABASE_URL" -c "SELECT version();" 
```

Use Neon, not Render Postgres. Render’s free database expires after 30 days and will delete your evidence mid-project.

0.7 — LLM API key (10 min)

Either: - `console.anthropic.com` → API keys. Budget roughly ₹500 for the whole project. - `aistudio.google.com` → free tier, zero cost, slightly weaker reasoning on the adjudication task.

Smoke test:

```
curl -s https://api.anthropic.com/v1/messages \
-H "x-api-key: $ANTHROPIC_API_KEY" \
-H "anthropic-version: 2023-06-01" \
-H "content-type: application/json" \
-d '{"model": "claude-sonnet-4-6", "max_tokens": 32,
  "messages": [{"role": "user", "content": "reply with OK"}]}' | jq
-r '.content[0].text'
```

0.8 — Phase 0 gate

All five must pass before you write a line of application code:

```
ffmpeg -version          # laptop: works, libx264
present
ssh ubuntu@<VM_IP> 'echo connected' # laptop: connects without
password
ssh ubuntu@<VM_IP> 'df -h /data'    # VM: 100 GB mounted
psql "$DATABASE_URL" -c 'SELECT 1;' # VM: returns a row
curl ... anthropic ...             # VM: returns OK
```

Failure modes in Phase 0

Symptom	Cause	Fix
“Out of capacity” on instance create	ARM demand in that region	Retry hourly, or change region, or use Hetzner
Port open in OCI but connection refused	Instance iptables still blocking	Run the <code>iptables -I INPUT 6</code> commands
<code>pip install av</code> fails compiling	Missing ffmpeg dev headers	<code>sudo apt install libavformat-dev libavcodec-dev libavdevice-dev</code>
<code>psql</code> hangs	Neon project asleep	Open the Neon console once to wake it
Volume unmounts on reboot	Missing <code>_netdev,nofail</code> in fstab	Re-add the fstab line exactly as written

Phase 1 — Camera Simulation and Datasets

Objective: Twelve live RTSP endpoints carrying real surveillance footage, split into two scene cohorts.

Budget: 3 hours.

1.1 — Install MediaMTX on the VM (15 min)

```
cd /tmp
wget
https://github.com/bluenvoron/mediamtx/releases/latest/download/mediamtx_linux_arm64v8.tar.gz
tar xzf mediamtx_linux_arm64v8.tar.gz
sudo mv mediamtx /usr/local/bin/
sudo mkdir -p /etc/mediamtx && sudo mv mediamtx.yml /etc/mediamtx/
```

Edit `/etc/mediamtx/mediamtx.yml` and set these keys:

```
logLevel: info
rtspAddress: :8554
protocols: [tcp]
api: yes
apiAddress: :9997
paths:
  all_others:
```

Force TCP. RTSP over UDP through a cloud NAT gateway drops packets unpredictably and will corrupt your measurements with noise that has nothing to do with your system.

Run it as a service:

```
sudo tee /etc/systemd/system/mediamtx.service > /dev/null <<'EOF'
[Unit]
Description=MediaMTX
After=network.target
[Service]
ExecStart=/usr/local/bin/mediamtx /etc/mediamtx/mediamtx.yml
Restart=always
User=ubuntu
[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload && sudo systemctl enable --now mediamtx
sudo systemctl status mediamtx --no-pager
```

1.2 — Download the datasets (45 min, mostly waiting)

Everything downloads directly onto the VM's block volume. **Never download to your laptop.** A 512 GB HDD does not need 20 GB of surveillance video on it, and the VM has a far faster link.

```
mkdir -p /data/datasets && cd /data/datasets

# ShanghaiTech Campus: 13 distinct scenes, your domain-shift cohort source
wget -c https://svip-lab.github.io/dataset/campus_dataset.html # follow link to actual archive
unzip shanghaitech.zip -d shanghaitech/

# UR Fall Detection: RGB sequences with ground truth
mkdir -p urfall && cd urfall
for i in $(seq -w 1 30); do
  wget -c "http://fenix.ur.edu.pl/~mkepski/ds/data/fall-$i-cam0-rgb.zip"
done
cd ..
```

For UCF-Crime, **do not pull the full 128 hours.** Take 30 to 50 clips from a Kaggle or Hugging Face mirror across four anomaly classes. Total target footprint for all datasets: **under 15 GB.**

Verify the scene split you will use as cohorts:

```
ls /data/datasets/shanghaitech/training/videos/ | head -20
du -sh /data/datasets/*
```

Why ShanghaiTech is the critical one: it has 13 distinct scenes with different lighting and camera angles. Train on scenes 01 to 08, deploy to 09 to 13, and you get a genuine domain shift you measured rather than one you speculated about. That is the whole of Question 2.2.

1.3 — The camera spawn script (45 min)

```

mkdir -p ~/video-intel/sim
cat > ~/video-intel/sim/spawn.sh <<'EOF'
#!/usr/bin/env bash
# Publishes dataset clips as live RTSP streams, split into two site cohorts.
set -euo pipefail

MTX="rtsp://127.0.0.1:8554"
SRC="/data/datasets/shanghaitech/training/videos"
LOGDIR="/data/simlogs"
mkdir -p "$LOGDIR"

spawn () {
    local file="$1" cam="$2"
    ffmpeg -hide_banner -loglevel warning \
        -re -stream_loop -1 -i "$file" \
        -c:v libx264 -preset ultrafast -tune zerolatency \
        -b:v 900k -maxrate 900k -bufsize 1800k \
        -g 50 -keyint_min 50 -sc_threshold 0 \
        -pix_fmt yuv420p -an \
        -f rtsp -rtsp_transport tcp "$MTX/$cam" \
        > "$LOGDIR/$cam.log" 2>&1 &
    echo "$cam -> $(basename "$file") pid $!"
}

i=0
for f in "$SRC"/*.avi; do
    i=$((i+1))
    if [ $i -le 6 ]; then spawn "$f" "$(printf 'a_cam%02d' $i)"
    elif [ $i -le 12 ]; then spawn "$f" "$(printf 'b_cam%02d' $((i-6)))"
    else break; fi
done

echo "spawned $i cameras"
EOF

chmod +x ~/video-intel/sim/spawn.sh
~/video-intel/sim/spawn.sh

```

Every flag in that ffmpeg line matters. Understand them, because you will be asked.

Flag	Why
<code>-re</code>	Paces at real time. Without it ffmpeg blasts the whole file in seconds and nothing about the test is realistic
<code>-stream_loop -1</code>	Infinite loop, so the camera never “ends”
<code>-g 50 -keyint_min 50 -sc_threshold 0</code>	Forces a keyframe every 50 frames (2 s at 25 fps). This is what makes keyframe-aligned segmenting possible. Without it, segment boundaries drift and your durations become inconsistent
<code>-b:v 900k -maxrate -bufsize</code>	Constrains bitrate so your measured bandwidth is a controlled variable, not a property of whichever clip happened to load
<code>-preset ultrafast</code>	ARM cores are weak. This keeps 12 encoders under control
<code>-rtsp_transport tcp</code>	Matches the server config; avoids UDP loss noise

You transcode here because dataset files have wildly inconsistent encodings. **Note this in the write-up:** real cameras deliver a constrained stream natively, so this transcode step exists only in simulation and does not exist in the production cost model.

1.4 — Stop script and verification (15 min)

```

cat > ~/video-intel/sim/stop.sh <<'EOF'
#!/usr/bin/env bash
kill -f "rtsp://127.0.0.1:8554" || true
echo "cameras stopped"
EOF
chmod +x ~/video-intel/sim/stop.sh

```

Verify from the VM:

```
curl -s localhost:9997/v3/paths/list | jq -r '.items[].name'
```

Verify from your laptop, which proves the whole path works end to end:

```

ffprobe -v error -rtsp_transport tcp -show_streams \
    rtsp://<VM_IP>:8554/a_cam01 2>&1 | grep -E
"codec_name|width|height|r_frame_rate"

```

1.5 — Zone definitions (30 min)

Zones live in YAML, not in a drawing tool. Building an editor costs eight hours and demonstrates nothing.

```
cat > ~/video-intel/edge/zones.yaml <<'EOF'
a_cam01:
- name: entrance
  type: dwell
  dwell_seconds: 25
  polygon: [[0.30,0.40],[0.70,0.40],[0.70,0.95],[0.30,0.95]]
- name: doorway
  type: tripwire
  line: [[0.10,0.55],[0.90,0.55]]
  direction: down
a_cam02:
- name: corridor
  type: dwell
  dwell_seconds: 30
  polygon: [[0.20,0.30],[0.80,0.30],[0.80,0.90],[0.20,0.90]]
EOF
```

Coordinates are normalised 0 to 1, so a zone survives a resolution change. This is not pedantry; hardcoded pixel zones are a real production bug that breaks whenever a camera’s sub-stream profile changes.

To draw them, grab a frame and open it in any image viewer:

```
ffmpeg -rtsp_transport tcp -i rtsp://127.0.0.1:8554/a_cam01 \
-frames:v 1 /tmp/a_cam01.png
```

Divide your pixel coordinates by width and height. Fifteen minutes of arithmetic beats eight hours of canvas code.

1.6 — Phase 1 gate

```
# 12 paths listed
curl -s localhost:9997/v3/paths/list | jq '.items | length'

# each is actually publishing
curl -s localhost:9997/v3/paths/list | jq -r '.items[] | "\(.name) \
(.ready)'"

# CPU sanity: 12 encoders should sit well under saturation
top -bn1 | head -15
```

If `ready` is `false` for any path, check `/data/simlogs/<cam>.log`.

Failure modes in Phase 1

Symptom	Cause	Fix
ffmpeg exits immediately	Codec not supported in source file	Test one file manually with <code>-loglevel debug</code>
All 12 spawn then die after ~30 s	CPU saturation	Reduce to 8 cameras, or <code>-s 640x360</code> to downscale
<code>ready: false</code> in the API	Publish failed	Read <code>/data/simlogs/<cam>.log</code> , usually a path or codec error
<code>ffprobe</code> works locally, fails from laptop	Port 8554 blocked	Both OCI security list and instance iptables
Streams stutter every few seconds	UDP transport still in use	Confirm <code>protocols: [tcp]</code> and restart <code>mediamtx</code>

Phase 2 — Telemetry Schema and Edge Agent

Objective: Recording that survives outages, a manifest that proves it, an outbox that never loses an event, and a log line for every single thing that happens.

Budget: 8 hours. This is the largest phase and the one that carries most of your marks.

2.1 — The telemetry schema (2 hours, highest leverage work in the project)

Write this before anything else. Every one of your fourteen measurements is a `jq` one-liner if this exists from hour one, and unrecoverable if it does not.

```
cat > ~/video-intel/common/telem.py <<'EOF'
import json, os, sys, time, uuid, threading

RUN_ID = os.environ.get("RUN_ID") or str(uuid.uuid4())[:8]
_lock = threading.Lock()
_fh = None

def init(logfile):
    global _fh
    os.makedirs(os.path.dirname(logfile), exist_ok=True)
    _fh = open(logfile, "a", buffering=1)

def emit(stage, **kw):
    rec = {"ts": round(time.time(), 3), "run_id": RUN_ID, "stage":
stage}
    rec.update(kw)
    line = json.dumps(rec, separators=(",", ":"))
    with _lock:
        if _fh:
            _fh.write(line + "\n")
        else:
            sys.stdout.write(line + "\n")

class Timer:
    def __init__(self, stage, **kw):
        self.stage, self.kw = stage, kw
    def __enter__(self):
        self.t0 = time.perf_counter(); return self
    def __exit__(self, *a):
        emit(self.stage, latency_ms=round((time.perf_counter()-
self.t0)*1000, 2), **self.kw)
EOF
```

Field contract. Every record carries `ts`, `run_id`, `stage`. Beyond that, per stage:

Stage	Required fields	Feeds measurement
segment_write	camera_id, site_id, seq, bytes, duration_s	M1 bitrate, M14 saturation
stream_health	camera_id, site_id, fps_effective, reconnects	M9, silent-failure detection
completeness	camera_id, site_id, expected, actual, ratio	M8, the Q1.3 detector
motion_gate	camera_id, passed, frames_in	M3 rejection rate
detect	camera_id, latency_ms, n_det, model_ver	M4 throughput
track	camera_id, n_tracks	M10
candidate	camera_id, site_id, rule, track_id, dwell_s	M5 event rate
outbox	site_id, depth, oldest_age_s	M8 recovery curve
upload	site_id, n_events, bytes, ok	M6 payload size, M7
verdict	candidate_id, is_false_alarm, confidence, latency_ms	M12 FP ladder
policy_change	camera_id, field, old, new, reason	Feedback loop evidence
agent_incident	detected_at, root_cause, latency_s	M13

The single rule: if a number will appear in your written answers, it must be emitted here first. Never compute it by hand at the end.

2.2 — SQLite schema (30 min)

```
cat > ~/video-intel/edge/schema.sql <<'EOF'
CREATE TABLE IF NOT EXISTS segments (
  camera_id TEXT NOT NULL,
  seq      INTEGER NOT NULL,
  path     TEXT NOT NULL,
  start_ts REAL NOT NULL,
  end_ts   REAL,
  bytes    INTEGER,
  uploaded INTEGER DEFAULT 0,
  PRIMARY KEY (camera_id, seq)
);
CREATE INDEX IF NOT EXISTS idx_seg_up ON segments(camera_id,
uploaded);

CREATE TABLE IF NOT EXISTS outbox (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,
  idem_key TEXT UNIQUE NOT NULL,
  kind    TEXT NOT NULL,
  payload TEXT NOT NULL,
  created REAL NOT NULL,
  attempts INTEGER DEFAULT 0,
  sent    INTEGER DEFAULT 0
);
CREATE INDEX IF NOT EXISTS idx_out_pending ON outbox(sent, id);

CREATE TABLE IF NOT EXISTS policies (
  camera_id TEXT NOT NULL,
  zone      TEXT NOT NULL,
  field     TEXT NOT NULL,
  value     TEXT NOT NULL,
  version   INTEGER NOT NULL,
  PRIMARY KEY (camera_id, zone, field)
);
EOF
```

Enable WAL mode. Without it your writer and drain loop will deadlock under load, and you will waste two hours diagnosing it.

```
conn.execute("PRAGMA journal_mode=WAL")
conn.execute("PRAGMA synchronous=NORMAL")
```

2.3 — The recorder (2 hours)

One ffmpeg subprocess per camera. **Copy, never decode.**

```
ffmpeg -rtsp_transport tcp -i rtsp://127.0.0.1:8554/a_cam01 \
-c copy -f segment -segment_time 10 \
-segment_format mp4 -reset_timestamps 1 -strftime 1 \
/data/seg/a_cam01/%Y%m%d_%H%M%S.mp4
```

`-c copy` is the entire cost argument of your architecture. Decoding costs roughly a full core per stream; demux-and-copy costs about two percent of one. **This one flag is why a modest box holds 100+ cameras, and it is why using `cv2.VideoCapture` for recording is architecturally wrong.**

A watcher thread does the manifest bookkeeping:

```
# edge/recorder.py (structure, not full listing)
import subprocess, threading, time, sqlite3, os, glob
from common import telem

def watch(camera_id, site_id, segdir, conn):
    seen, seq = set(), 0
    while True:
        for p in sorted(glob.glob(f"{segdir}/*.mp4")):
            if p in seen:
                continue
            # skip the file currently being written
            if time.time() - os.path.getmtime(p) < 2:
                continue
            seen.add(p); seq += 1
            size = os.path.getsize(p)
            conn.execute(
                "INSERT OR IGNORE INTO
segments(camera_id,seq,path,start_ts,bytes) VALUES(?,?,?,?,?)",
                (camera_id, seq, p, os.path.getmtime(p), size))
            conn.commit()
            telem.emit("segment_write", camera_id=camera_id,
            site_id=site_id,
                        seq=seq, bytes=size, duration_s=10)
        time.sleep(2)
```

That `segment_write` line is **measurement M1**. Bitrate per camera is now `bytes / duration_s * 8 / 1000` kbps, computed over thousands of real samples rather than guessed.

Restart supervision: wrap each ffmpeg in a loop with exponential backoff plus jitter.

```
delay = min(60, 2 ** attempt) + random.uniform(0, 3)
```

The jitter is not optional. When a site's power restores, 200 cameras reconnecting simultaneously is a thundering herd that will saturate the uplink and take down your ingestion tier. You will demonstrate exactly this in Phase 6.

2.4 — Expected-versus-actual accounting (1 hour)

Thirty lines of code, and it is your entire answer to Question 1.3.

```
def completeness_loop(camera_id, site_id, conn, window=60,
seg_s=10):
    while True:
        time.sleep(window)
        expected = window // seg_s
        cutoff = time.time() - window
        actual = conn.execute(
            "SELECT COUNT(*) FROM segments WHERE camera_id=? AND
start_ts>?",
            (camera_id, cutoff)).fetchone()[0]
        ratio = actual / expected if expected else 0
        telem.emit("completeness", camera_id=camera_id,
        site_id=site_id,
                    expected=expected, actual=actual,
        ratio=round(ratio, 3))
```

Why this matters more than anything else in the observability story: a camera that disconnects is trivially detectable. A camera that stays connected while quietly dropping 30 percent of its frames looks perfectly healthy on every naive dashboard. That is precisely the customer complaint in Question 1.3, and this ratio is the only thing that catches it.

2.5 — The durable outbox (2 hours)

Nothing is ever emitted from memory. Write to SQLite first, then attempt the network.

```
def enqueue(conn, kind, payload, idem_key):
    conn.execute("INSERT OR IGNORE INTO
outbox(idem_key,kind,payload,created) VALUES(?,?,?,?)",
                (idem_key, kind, json.dumps(payload), time.time()))
    conn.commit()

def drain(conn, site_id, api, batch=20, rate_cap_bps=None):
    while True:
        rows = conn.execute(
            "SELECT id,idem_key,kind,payload FROM outbox WHERE
sent=0 ORDER BY id LIMIT ?",
            (batch,)).fetchall()
        if not rows:
            time.sleep(2); continue
        body = [{"idem_key": r[1], "kind": r[2], "payload":
json.loads(r[3])} for r in rows]
        raw = json.dumps(body)
        try:
            with telem.Timer("upload", site_id=site_id,
n_events=len(rows), bytes=len(raw)):
                r = requests.post(f"{api}/events", data=raw,
timeout=10,
                                headers={"content-type":
"application/json"})
                if r.status_code < 300:
                    conn.executemany("UPDATE outbox SET sent=1 WHERE
id=?", [(x[0],) for x in rows])
                    conn.commit()
                except Exception:
                    time.sleep(5)
            depth = conn.execute("SELECT COUNT(*) FROM outbox WHERE
sent=0").fetchone()[0]
            oldest = conn.execute("SELECT MIN(created) FROM outbox WHERE
sent=0").fetchone()[0]
            telem.emit("outbox", site_id=site_id, depth=depth,
                        oldest_age_s=round(time.time()-oldest,1) if
oldest else 0)
            if rate_cap_bps:
                time.sleep(len(raw) * 8 / rate_cap_bps)
```

Three properties you must be able to defend:

1. **Ordered.** `ORDER BY id` means backfill drains oldest first. Out-of-order recovery is a real bug that produces incoherent event timelines.
2. **Idempotent.** The `idem_key` unique constraint on both ends means a retry after a timeout that actually succeeded does not duplicate.
3. **Rate capped.** `rate_cap_bps` stops backfill from saturating a 5 Mbps site link and starving live events. Priority queueing in one line.

The `outbox` telemetry line **is measurement M8**, the outage recovery curve.

2.6 — Agent entry point (30 min)

```
cat > ~/video-intel/edge/run.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
SITE=$1 # a or b
export RUN_ID=${RUN_ID:-$(date +%s | tail -c 6)}
source ~/vi-env/bin/activate
cd ~/video-intel
python -m edge.agent \
--site "$SITE" \
--cameras "$(curl -s localhost:9997/v3/paths/list | jq -r
.items[.name | select(startswith("${SITE}_")]" | paste -sd,)" \
--db "/data/edge_${SITE}.db" \
--segdir "/data/seg/${SITE}" \
--log "/data/logs/edge_${SITE}.jsonl" \
--api "${CLOUD_API}"
EOF
chmod +x ~/video-intel/edge/run.sh
```

2.7 — Phase 2 gate

This gate is the most important in the document. It is the outage demo in miniature.

```
# 1. run both agents
./edge/run.sh a & ./edge/run.sh b &

# 2. segments appearing
watch -n2 'ls /data/seg/a/a_cam01 | wc -l'

# 3. manifest populated
sqlite3 /data/edge_a.db "SELECT camera_id, COUNT(*), SUM(bytes) FROM
segments GROUP BY 1;"

# 4. KILL the cloud API, watch the outbox grow
pkill -f uvicorn
watch -n2 'sqlite3 /data/edge_a.db "SELECT COUNT(*) FROM outbox
WHERE sent=0;"'

# 5. restore, watch it drain in order
uvicorn cloud.api:app --host 0.0.0.0 --port 8000 &
watch -n2 'sqlite3 /data/edge_a.db "SELECT COUNT(*) FROM outbox
WHERE sent=0;"'

# 6. prove zero loss
sqlite3 /data/edge_a.db "SELECT COUNT(*) FROM outbox WHERE sent=0;"
# must be 0
```

If step 6 returns anything but zero, stop and fix it before Phase 3. Everything downstream depends on this guarantee.

Failure modes in Phase 2

Symptom	Cause	Fix
database is locked	WAL not enabled	Add both PRAGMAs at connection open
Segments 0 bytes	Watcher reading the file being written	Keep the 2 s mtime guard
Segment durations drift from 10 s	Source GOP not aligned	Phase 1.3 -g 50 -sc_threshold 0
Outbox grows forever after recovery	Server not returning 2xx	Check the API log; the drain only marks sent on success
Duplicate events after a retry	Missing unique constraint server-side	Add a unique index on idem_key in Postgres too
Disk fills	No ring buffer eviction	Cron: delete segments older than N hours, oldest first

Phase 3 — The Inference Cascade

Objective: A cost cascade where each stage exists to make the next stage affordable, running on CPU only.

Budget: 7 hours.

3.1 — Export the model on Kaggle (1 hour)

Do not install PyTorch on the VM or the laptop. It is 2 GB, it will not help, and on 4 GB of RAM it will swap.

Create a Kaggle notebook (free, roughly 30 GPU hours per week):

```
!pip install rfdetr onnx onnxruntime
from rfdetr import RFDETRNano
m = RFDETRNano()
m.export(format="onnx", opset=17)
```

Then quantise to INT8, which roughly halves the file and speeds up ARM inference substantially:

```
from onnxruntime.quantization import quantize_dynamic, QuantType
quantize_dynamic("rf-detr-nano.onnx", "rf-detr-nano-int8.onnx",
                 weight_type=QuantType.QInt8)
```

Download the `.onnx` (roughly 15 to 30 MB) and copy it to the VM:

```
scp rf-detr-nano-int8.onnx ubuntu@<VM_IP>:/data/models/
```

Choose RF-DETR over Ultralytics YOLO deliberately, and say why in your write-up. RF-DETR ships under Apache 2.0, so it is usable in a commercial product without source disclosure. Ultralytics models are AGPL-3.0. Its DINOv2 backbone also transfers better under domain shift, which is directly relevant to Question 2.2. That is an architectural justification, not a preference.

3.2 — The motion gate (1 hour)

```
# edge/gate.py
import cv2, numpy as np
from common import telem

class MotionGate:
    def __init__(self, camera_id, min_area_frac=0.0015, scale=0.25):
        self.bs = cv2.createBackgroundSubtractorMOG2(history=300,
                                                    varThreshold=24, detectShadows=False)
        self.cam, self.min_area_frac, self.scale = camera_id,
        min_area_frac, scale
        self.k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))

    def passes(self, frame):
        small = cv2.resize(frame, None, fx=self.scale,
                           fy=self.scale)
        m = self.bs.apply(small)
        m = cv2.morphologyEx(m, cv2.MORPH_OPEN, self.k)
        frac = float(np.count_nonzero(m)) / m.size
        ok = frac > self.min_area_frac
        telem.emit("motion_gate", camera_id=self.cam, passed=ok,
                  motion_frac=round(frac, 5))
        return ok
```

Roughly 2 ms per frame on ARM. It rejects 85 to 95 percent of frames in static surveillance scenes.

This is the most important component in the entire cost argument. It is the reason CPU inference is viable at all on your hardware, and it is the same reason the architecture scales to 10,000 cameras. Your constraint forced the correct design. Say that explicitly.

Calibration: run for 30 minutes, then check the pass rate per camera:

```
jq -r 'select(.stage=="motion_gate") | "\(.camera_id) \(.passed)"'
/data/logs/edge_a.jsonl \
| sort | uniq -c
```

Target 5 to 15 percent pass. Above 20 percent, raise `min_area_frac`. Below 3 percent, lower it, because you are dropping real activity.

3.3 — Decode path with PyAV (2 hours)

```
# edge/decode.py
import av, time
from common import telem

def frames(rtsp_url, camera_id, target_fps=4):
    container = av.open(rtsp_url, options={
        "rtsp_transport": "tcp", "stimeout": "5000000"})
    stream = container.streams.video[0]
    stream.thread_type = "AUTO"
    src_fps = float(stream.average_rate or 25)
    stride = max(1, int(round(src_fps / target_fps)))
    n = 0
    for packet in container.demux(stream):
        for frame in packet.decode():
            n += 1
            if n % stride:
                continue
            pts_s = float(frame.pts * stream.time_base) if frame.pts
            is not None else None
            yield frame.to_ndarray(format="bgr24"), pts_s,
            time.time()
```

Two things here are load-bearing.

`stride` decimates in the decode loop, so you decode every frame but only process every Nth. If you want to skip decode entirely for skipped frames, gate on keyframes with `packet.is_keyframe`, at some accuracy cost.

`pts_s` is the presentation timestamp. **All temporal rules use this, never frame counts.** A rule expressed as “300 frames” fires at 10 seconds at 30 fps and 60 seconds at 5 fps. Effective fps varies with load, motion gate rate, and network conditions, so a frame-count rule produces different real-world behaviour under different conditions. That is a genuine production bug class, and mentioning it signals experience more than three paragraphs of architecture.

3.4 — Detection (1 hour)

```
# edge/detect.py
import onnxruntime as ort, numpy as np
from common import telem

class Detector:
    def __init__(self, path, size=384, conf=0.35):
        so = ort.SessionOptions()
        so.intra_op_num_threads = 2
        self.s = ort.InferenceSession(path, so, providers=
["CPUExecutionProvider"])
        self.inp = self.s.get_inputs()[0].name
        self.size, self.conf = size, conf
        self.ver = path.split("/")[-1]

    def __call__(self, frame, camera_id):
        import cv2
        h, w = frame.shape[:2]
        img = cv2.resize(frame, (self.size, self.size))[:, :, ::-1]
        x = (img.astype(np.float32) / 255.0).transpose(2,0,1)[None]
        with telem.Timer("detect", camera_id=camera_id,
model_ver=self.ver):
            out = self.s.run(None, {self.inp: x})
            boxes = self._post(out, w, h)
            telem.emit("detect_result", camera_id=camera_id,
n_det=len(boxes))
            return boxes
```

Set `intra_op_num_threads` explicitly. ONNX Runtime defaults to all cores, and with two agents plus twelve ffmpeg encoders on 2 OCPUs, thread oversubscription will make everything slower. Two threads per detector is right for this box.

Batch across cameras rather than running one session per camera. One session, a shared queue, and a small batch is dramatically more efficient than twelve independent sessions.

Expected on 2 ARM cores at 384px INT8: roughly 100 to 200 ms per frame. With a 90 percent gate, that supports 8 to 12 cameras with bursty behaviour. **Plan for 8.**

3.5 — Tracking (1 hour)

A minimal IoU tracker is about 80 lines and is entirely sufficient at 4 fps. Full ByteTrack is nicer but is not worth three hours here.

```
# edge/track.py (core logic)
class Track:
    def __init__(self, tid, box, ts):
        self.id, self.box = tid, box
        self.first_seen = self.last_seen = ts
        self.zone_entry = {}

class IouTracker:
    def __init__(self, iou_thr=0.3, max_age=3.0):
        self.tracks, self.next_id = [], 1
        self.iou_thr, self.max_age = iou_thr, max_age

    def update(self, boxes, ts):
        for t in self.tracks:
            best, bi = 0, None
            for i, b in enumerate(boxes):
                v = iou(t.box, b)
                if v > best: best, bi = v, i
            if best > self.iou_thr:
                t.box, t.last_seen = boxes.pop(bi), ts
        for b in boxes:
            self.tracks.append(Track(self.next_id, b, ts));
        self.next_id += 1
        self.tracks = [t for t in self.tracks if ts - t.last_seen <
            self.max_age]
        return self.tracks
```

`max_age=3.0` in **seconds**, not frames. Same reason as before.

3.6 — Temporal rules (1 hour)

```
# edge/rules.py
from common import telem

def evaluate(tracks, zones, camera_id, site_id, ts, policy,
emit_candidate):
    for t in tracks:
        cx, cy = center_norm(t.box)
        for z in zones:
            inside = point_in_poly((cx, cy), z["polygon"]) if
z["type"] == "dwelling" else False
            if inside:
                t.zone_entry.setdefault(z["name"], ts)
                dwell = ts - t.zone_entry[z["name"]]
                thr = policy.get(camera_id, {}).get(z["name"],
{}).get(
                    "dwell_seconds", z["dwell_seconds"])
                if dwell > thr and not t.fired.get(z["name"]):
                    t.fired[z["name"]] = True
                    telem.emit("candidate", camera_id=camera_id,
site_id=site_id,
                                rule="loitering", zone=z["name"],
                                track_id=t.id,
dwell_s=round(dwell,1))
                    emit_candidate(t, z, dwell)
            else:
                t.zone_entry.pop(z["name"], None)
```

Note `policy.get(...)` before falling back to the YAML default. **That lookup is the receiving end of your fast feedback loop.** The adjudicator will write into `policies`, the agent polls it every 30 seconds, and this line applies it. Closing the loop is that cheap once the structure exists.

3.7 — Phase 3 gate

```
# gate rejection rate is in range
jq -r 'select(.stage=="motion_gate")|.passed'
/data/logs/edge_a.jsonl | sort | uniq -c

# detection latency distribution
jq -r 'select(.stage=="detect")|.latency_ms' /data/logs/edge_a.jsonl
\
| sort -n | awk '{a[NR]=$1} END {print
"p50",a[int(NR*0.5)],"p99",a[int(NR*0.99)]}'

# candidates are being produced
jq -r 'select(.stage=="candidate")|.camera_id'
/data/logs/edge_a.jsonl | sort | uniq -c

# candidates reached the outbox
sqlite3 /data/edge_a.db "SELECT kind, COUNT(*) FROM outbox GROUP BY
1;"
```

You should see between 20 and 80 candidates per camera per day. Far more means your thresholds are too loose, which is fine, because the adjudicator exists precisely to fix that.

Failure modes in Phase 3

Symptom	Cause	Fix
ONNX session load kills the process	OOM on 12 GB with 12 encoders	Add swap (Phase 0.5), reduce camera count
Detection p99 above 800 ms	Thread oversubscription	Set <code>intra_op_num_threads=2</code>
Gate passes 100 percent of frames	MOG2 not warmed up	Discard the first 100 frames per camera
Zero candidates after an hour	Zones wrong or threshold too high	Draw a frame, verify normalised coords
PyAV hangs on reconnect	No socket timeout	Set <code>stimeout</code> in the open options
Track IDs churn constantly	4 fps too slow for the scene	Raise to 6 fps on that camera, or raise <code>max_age</code>

Phase 4 — Cloud Plane and the Adjudicator

Objective: Idempotent ingestion, and the second-stage adjudicator that is your primary differentiator.

Budget: 6 hours.

4.1 — Postgres schema on Neon (45 min)

```
CREATE TABLE sites (
  site_id TEXT PRIMARY KEY, name TEXT, region TEXT);

CREATE TABLE cameras (
  camera_id TEXT PRIMARY KEY,
  site_id TEXT REFERENCES sites(site_id),
  scene_id TEXT,
  cohort TEXT);          -- 'train' or 'holdout' <-- the Q2.2
                          lever

CREATE TABLE candidates (
  id BIGSERIAL PRIMARY KEY,
  idem_key TEXT UNIQUE NOT NULL,
  camera_id TEXT REFERENCES cameras(camera_id),
  site_id TEXT,
  rule TEXT, zone TEXT, track_id INT,
  dwell_s REAL, ts REAL,
  thumbs TEXT,
  created TIMESTAMPTZ DEFAULT now());

CREATE TABLE verdicts (
  candidate_id BIGINT REFERENCES candidates(id),
  is_false_alarm BOOLEAN,
  confidence INT,
  reasoning TEXT,
  model TEXT,
  latency_ms REAL,
  created TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (candidate_id));

CREATE TABLE telemetry (
  ts REAL, run_id TEXT, stage TEXT, camera_id TEXT,
  site_id TEXT, payload JSONB);
CREATE INDEX ON telemetry(stage, ts);
CREATE INDEX ON telemetry(camera_id, ts);

CREATE TABLE policies (
  camera_id TEXT, zone TEXT, field TEXT, value TEXT,
  version INT, reason TEXT, updated TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (camera_id, zone, field));

CREATE TABLE models (
  model_id TEXT PRIMARY KEY, version TEXT, trained_on TEXT, created
  TIMESTAMPTZ);
CREATE TABLE deployments (
  model_id TEXT REFERENCES models(model_id),
  site_id TEXT REFERENCES sites(site_id),
  stage TEXT, -- 'shadow' | 'canary' | 'production'
  started TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (model_id, site_id));
```

The `cohort` column on `cameras` is the single most valuable field in the schema. Mark scenes 01 to 08 `train` and 09 to 13 `holdout`. Every accuracy query then splits by cohort for free, and Question 2.2 becomes a report of findings rather than speculation.

The `deployments` table joining `models` to `sites` is what lets you say “accuracy dropped at these three sites, and here is exactly which model version each was running.” That sentence is worth a full rubric point.

4.2 — FastAPI ingestion (1.5 hours)

```

# cloud/api.py
from fastapi import FastAPI, Request
import json, psycopg2, os
app = FastAPI()
db = psycopg2.connect(os.environ["DATABASE_URL"])

@app.post("/events")
async def events(req: Request):
    batch = await req.json()
    cur = db.cursor()
    for item in batch:
        k, kind, p = item["idem_key"], item["kind"], item["payload"]
        if kind == "candidate":
            cur.execute("""INSERT INTO candidates

(idem_key,camera_id,site_id,rule,zone,track_id,dwell_s,ts,thumbs)
VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s)
ON CONFLICT (idem_key) DO NOTHING""",
            (k, p["camera_id"], p["site_id"], p["rule"],
p["zone"],
p["track_id"], p["dwell_s"], p["ts"],
json.dumps(p.get("thumbs"))))
        elif kind == "telemetry":
            cur.execute("""INSERT INTO
telemetry(ts,run_id,stage,camera_id,site_id,payload)
VALUES (%s,%s,%s,%s,%s,%s)""",
            (p["ts"], p["run_id"], p["stage"], p.get("camera_id"),
p.get("site_id"), json.dumps(p)))
    db.commit()
    return {"ok": True, "n": len(batch)}

@app.get("/policies/{camera_id}")
def policies(camera_id: str):
    cur = db.cursor()
    cur.execute("SELECT zone,field,value,version FROM policies WHERE
camera_id=%s",
        (camera_id,))
    return [{"zone":z,"field":f,"value":v,"version":n} for z,f,v,n
in cur.fetchall()]

```

`ON CONFLICT (idem_key) DO NOTHING` is the server half of your at-least-once contract. Combined with the outbox's unique key, a retry after a timeout that actually succeeded is harmless. **Say “at-least-once with idempotent writes” in your answer, not “exactly-once,” which does not exist.**

4.3 — The adjudicator (3 hours, your differentiator)

This is where the marks are. Give it the full three hours.

```
# cloud/adjudicator.py
import base64, json, os, time, requests, psycpg2
from common import telem

SYSTEM = """You are a security operations analyst reviewing an
automated alert.
A computer vision system flagged a candidate event. Your job is to
decide whether
it is a genuine security concern or a false alarm, using the visual
context.

Be conservative about dismissing genuine threats and aggressive
about dismissing
benign activity. A person waiting for transport, using a phone, or
having a
conversation is NOT loitering. A person repeatedly approaching and
retreating
from a restricted boundary, obscuring their face, or testing doors
IS.

Respond with ONLY a JSON object, no preamble, no markdown fences:
{"is_false_alarm": bool,
 "confidence": 0-100,
 "reasoning": "one or two sentences",
 "suggested_policy": {"dwell_seconds": int} or null}"""

def adjudicate(cand, frames_b64):
    content = [{"type": "image", "source": {"type": "base64",
        "media_type": "image/jpeg", "data": b}} for b in
frames_b64]
    content.append({"type": "text", "text":
        f"Rule fired: {cand['rule']} in zone '{cand['zone']}'".
        f"Dwell time: {cand['dwell_s']}s. Camera:
{cand['camera_id']}. "
        f"Three frames span the event window. Assess."})
    t0 = time.perf_counter()
    r = requests.post("https://api.anthropic.com/v1/messages",
        headers={"x-api-key": os.environ["ANTHROPIC_API_KEY"],
            "anthropic-version": "2023-06-01", "content-
type": "application/json"},
        json={"model": "claude-sonnet-4-6", "max_tokens": 300,
            "system": SYSTEM, "messages":
[{"role": "user", "content": content}]},
        timeout=45)
    lat = (time.perf_counter()-t0)*1000
    txt = r.json()["content"][0]
[ "text"].strip().strip("`").removeprefix("json").strip()
    v = json.loads(txt)
    telem.emit("verdict", candidate_id=cand["id"],
        is_false_alarm=v["is_false_alarm"],
        confidence=v["confidence"],
        latency_ms=round(lat,1))
    return v
```

Extract three frames at the event window start, middle, and end. Static context is the whole point; one frame cannot distinguish waiting from casing.

```
ffmpeg -ss <t> -i /data/seg/a/a_cam01/<segment>.mp4 -frames:v 1 -q:v
6 -vf scale=640:-1 /tmp/f.jpg
base64 -w0 /tmp/f.jpg
```

Scale to 640px and use `-q:v 6`. Full resolution wastes tokens without improving the judgement.

Why this stage is architecturally significant, and this is what you say in the demo: it breaks the precision-recall trade-off asymmetrically. Normally cutting false alarms by 80 percent costs you real detections, and Question 2.3 explicitly asks you to quantify that loss. A second-stage adjudicator lets you run the CV stage at deliberately high recall and low precision, then recover precision downstream without touching recall. Your answer becomes: “false alarms down 87 percent, true events lost 2 percent, because the filtering moved to a stage that reasons about context rather than pixels.”

Cost discipline. Adjudicating every detection costs roughly 400 times more than adjudicating only rule-escalated candidates. Compute both from your own logs and state the ratio. That is the same cascade principle at every tier: cheap and always-on in front, expensive and rare behind.

4.4 — The fast policy loop (1 hour)

```
def maybe_adjust_policy(db, camera_id, zone, window_hours=24,
min_n=8, fp_thresh=0.8):
    cur = db.cursor()
    cur.execute("""
        SELECT COUNT(*) FILTER (WHERE v.is_false_alarm), COUNT(*)
        FROM candidates c JOIN verdicts v ON v.candidate_id=c.id
        WHERE c.camera_id=%s AND c.zone=%s AND c.created > now() -
interval '%s hours'
    """, (camera_id, zone, window_hours))
    fp, n = cur.fetchone()
    if n >= min_n and fp / n >= fp_thresh:
        cur.execute("""
            INSERT INTO
policies(camera_id,zone,field,value,version,reason)
VALUES (%s,%s,'dwell_seconds',
        (SELECT COALESCE(MAX(value::int),25)*1.5 FROM
policies
            WHERE camera_id=%s AND zone=%s)::text, 1,
        %s)
        ON CONFLICT (camera_id,zone,field)
        DO UPDATE SET value=(policies.value::int*1.5)::text,
            version=policies.version+1,
            reason=EXCLUDED.reason, updated=now()
    """, (camera_id, zone, camera_id, zone,
        f"{fp}/{n} false in {window_hours}h"))
    db.commit()
    telem.emit("policy_change", camera_id=camera_id, zone=zone,
        field="dwell_seconds", reason=f"{fp}/{n} false")
```

The edge polls `GET /policies/{camera_id}` every 30 seconds. **Polling is completely adequate at this scale**; do not build a push channel for twelve cameras.

The distinction you must articulate: the fast loop changes *configuration* and completes in seconds. The slow loop changes *model weights* and completes in weeks. Conflating them is the common mistake, and separating them explicitly is a rubric point.

4.5 — Phase 4 gate

```
SELECT COUNT(*) FROM candidates;
SELECT COUNT(*) FROM verdicts;
SELECT is_false_alarm, COUNT(*) FROM verdicts GROUP BY 1;
SELECT c.camera_id, COUNT(*) FILTER (WHERE
v.is_false_alarm)::float/COUNT(*) AS fp_rate
FROM candidates c JOIN verdicts v ON v.candidate_id=c.id GROUP BY
1 ORDER BY 2 DESC;
SELECT * FROM policies;
```

The last query must return at least one row after a few hours of running. **If policies is empty, your feedback loop is not closed** and Question 2.3 loses its best evidence.

Failure modes in Phase 4

Symptom	Cause	Fix
JSON parse errors from the model	Markdown fences in output	Strip backticks and a leading <code>json</code> , as shown
Adjudicator latency above 20 s	Images too large	Scale to 640px, <code>-q:v6</code>
Duplicate candidates in Postgres	Missing unique index	<code>CREATE UNIQUE INDEX ON candidates(idem_key)</code>
Neon connection drops	Idle timeout	Reconnect on <code>OperationalError</code> , or use a pool
Every verdict says false alarm	Prompt too permissive	Add explicit positive examples to the system prompt
Policy never changes	<code>min_n</code> too high for your volume	Lower to 5 for the demo, note it in the write-up

Phase 5 — Observability

Objective: Real dashboards and real alert rules, configured rather than built.

Budget: 3 hours. **Do not build this in React.** Three hours of Grafana configuration beats twenty hours of custom UI and answers the question better.

5.1 — Metrics endpoint on the edge (45 min)

```
# edge/metrics.py
from prometheus_client import Counter, Gauge, Histogram,
start_http_server

SEG = Counter('segments_written_total', 'segments',
['camera_id','site_id'])
BYTES = Counter('segment_bytes_total', 'bytes',
['camera_id','site_id'])
COMPL = Gauge('stream_completeness_ratio', 'actual/expected',
['camera_id','site_id'])
FPS = Gauge('stream_fps_effective', 'fps', ['camera_id'])
GATE = Counter('motion_gate_total', 'frames',
['camera_id','passed'])
DET = Histogram('detect_latency_seconds', 'detect', ['camera_id'],
buckets=(.05,.1,.2,.4,.8,1.6))
OUTQ = Gauge('outbox_depth', 'pending', ['site_id'])
OUTA = Gauge('outbox_oldest_age_seconds', 'age', ['site_id'])
CAND = Counter('candidates_total', 'candidates',
['camera_id','rule'])

start_http_server(9101)
```

Call these from the same places you call `telem.emit`. **Both, always.** Prometheus is for the live dashboard and alerting; JSONL is for the offline analysis that produces your written answers. They serve different jobs and you need both.

5.2 — Prometheus (30 min)

```
cat > /data/prom.yml <<'EOF'
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: edge
    static_configs:
      - targets: ['host.containers.internal:9101']
        labels: {site: a}
      - targets: ['host.containers.internal:9102']
        labels: {site: b}
rule_files:
  - /etc/prometheus/alerts.yml
EOF

cat > /data/alerts.yml <<'EOF'
groups:
  - name: ingestion
    rules:
      - alert: StreamIncomplete
        expr: stream_completeness_ratio < 0.95
        for: 10m
        annotations:
          summary: "{{ $labels.camera_id }} completeness {{ $value }}"
      - alert: OutboxBacklog
        expr: outbox_depth > 200 and rate(outbox_depth[15m]) > 0
        for: 15m
      - alert: DetectLatencyHigh
        expr: histogram_quantile(0.99,
rate(detect_latency_seconds_bucket[5m])) > 0.8
        for: 10m
EOF

podman run -d --name prom -p 9090:9090 \
-v /data/prom.yml:/etc/prometheus/prometheus.yml:Z \
-v /data/alerts.yml:/etc/prometheus/alerts.yml:Z \
docker.io/prom/prometheus
```

The `:Z` suffix is mandatory on Fedora and on any SELinux-enforcing host. Without it the container cannot read the file and Prometheus starts with an empty config, silently.

`StreamIncomplete` at `< 0.95 for 10m` is your answer to “what should have alerted you before the customer complained.” **Point at this rule during the demo.**

5.3 — Grafana (1.5 hours)

```
podman run -d --name grafana -p 3000:3000 \
-v grafana-data:/var/lib/grafana:Z docker.io/grafana/grafana
```

Log in at http://<VM_IP>:3000 (admin/admin), add Prometheus at <http://host.containers.internal:9090>, and build **one** dashboard with six panels:

1. **Completeness heatmap** by camera. The hero panel. Silent degradation appears as a colour change here and nowhere else.
2. **Outbox depth** by site, stacked. This is the outage demo made visible.
3. **Detection latency** p50/p99.
4. **Motion gate pass rate** by camera.
5. **Candidates per hour** by rule.
6. **Effective fps** by camera.

Set the refresh to 10 s and the window to 6 h. Screenshot this dashboard during every chaos run; those images go straight into your submission.

Phase 6 — Chaos Runs and Measurement

Objective: Break your own system four times, keep the receipts, and extract every number your written answers need.

Budget: 5 hours. **This phase is where the marks actually are.**

6.1 — Toxiproxy setup (30 min)

```
podman run -d --name toxiproxy -p 8474:8474 -p 9000:9000 \
ghcr.io/shopify/toxiproxy

curl -X POST localhost:8474/proxies -d '{
"name":"cloud","listen":"0.0.0.0:9000","upstream":"127.0.0.1:8000"}'
```

Point site A's agent at <http://127.0.0.1:9000> and site B directly at [:8000](http://127.0.0.1:8000). **Now you can break one site while the other stays healthy**, which is exactly the cohort structure Question 1.3 needs.

6.2 — Run C1, full outage (1 hour)

```
export RUN_ID=c1_outage

# baseline for 10 minutes, then cut it
curl -X POST localhost:8474/proxies/cloud/toxics \
-d '{"type":"bandwidth","attributes":{"rate":0}}'

sleep 900 # 15 minutes of darkness

curl -X DELETE localhost:8474/proxies/cloud/toxics/bandwidth

# watch the drain
watch -n2 'sqlite3 /data/edge_a.db "SELECT COUNT(*) FROM outbox
WHERE sent=0;"'
```

Capture: the Grafana outbox-depth panel across the whole window, and the drain curve.

Measurement M8, outage duration versus backfill drain time:

```
jq -r 'select(.stage=="outbox" and .site_id=="a")| [.ts,.depth]|@tsv' \
/data/logs/edge_a.jsonl > /tmp/m8.tsv
```

Prove zero loss:

```
sqlite3 /data/edge_a.db \
"SELECT camera_id, COUNT(*), MAX(seq)-MIN(seq)+1 FROM segments
GROUP BY 1;"
```

Count must equal the sequence range. **A gap means a real bug, and finding it now is a gift.**

6.3 — Run C2, bandwidth cap (45 min)

```
export RUN_ID=c2_bwcap
curl -X POST localhost:8474/proxies/cloud/toxics \
-d '{"type":"bandwidth","attributes":{"rate":625}}' # KB/s = 5
Mbps
```

Show live events still flowing while backfill throttles. **That is priority queueing demonstrated, not asserted.**

6.4 — Run C3, thundering herd (45 min)

```
export RUN_ID=c3_herd
~/video-intel/sim/stop.sh
sleep 30
~/video-intel/sim/spawn.sh # all 12 return at once
```

Measure time to full recovery with and without reconnect jitter. **Run it both ways.** The comparison is your evidence, and the “without” run is the one that produces the interesting graph.

6.5 — Run C4, silent degradation (1 hour, the important one)

```
export RUN_ID=c4_silent
# throttle 4 of 12 cameras at the source, keeping them connected
for c in a_cam01 a_cam03 b_cam02 b_cam05; do
  pkill -f "$c"
  ffmpeg -re -stream_loop -1 -i <clip> -b:v 200k -r 8 \
  -f rtsp rtsp://127.0.0.1:8554/$c &
done
```

Do not look at the dashboard. Start a timer and let the alert fire on its own. Record how long it took.

This is the Question 1.3 scenario reproduced end to end: cameras that stay connected while quietly dropping data. Your **completeness** metric catches it. A naive up/down check never would.

6.6 — The measurement pass (1.5 hours)

Run every extraction and write results into **docs/evidence.md**.

```
L=/data/logs/*.jsonl

# M1 bitrate per camera (kbps)
jq -r 'select(.stage=="segment write")|
[.camera_id,.bytes,.duration_s]@tsv' $L \
| awk '{s[$1]+=$2; d[$1]+=$3} END {for (c in s) printf "%s %.1f kbps\n", c, s[c]*8/d[c]/1000}'

# M3 motion gate rejection rate
jq -r 'select(.stage=="motion_gate")|.[.camera_id,.passed]@tsv' $L \
| awk '{t[$1]++; if($2=="true") p[$1]++} END {for(c in t) printf "%s pass %.1f%%\n", c, 100*p[c]/t[c]}'

# M4 detection latency percentiles
jq -r 'select(.stage=="detect")|.latency_ms' $L | sort -n \
| awk '{a[NR]=$1} END {printf "p50 %.0f p95 %.0f p99 %.0f\n", a[int(NR*.5)],a[int(NR*.95)],a[int(NR*.99)]}'

# M5 candidates per camera per day
jq -r 'select(.stage=="candidate")|.camera_id' $L | sort | uniq -c

# M6 bytes per event upload
jq -r 'select(.stage=="upload")|.[.bytes,.n_events]@tsv' $L \
| awk '{b+=$1; n+=$2} END {printf "%.0f bytes/event\n", b/n}'

# M7 THE HEADLINE: continuous upload vs event-only
# = (M1 kbps * 86400) versus (M5 events/day * M6 bytes/event * 8)
```

M7 is the number you lead the demo with. It is the empirical basis of your entire architecture, and it is the one figure that makes the bandwidth inversion an observation rather than an assertion.

M11, per-cohort accuracy, is the Question 2.2 answer:

```
SELECT cam.cohort,
COUNT(*) FILTER (WHERE NOT v.is_false_alarm)::float /
COUNT(*) AS precision,
COUNT(*) AS n
FROM candidates c
JOIN verdicts v ON v.candidate_id = c.id
JOIN cameras cam ON cam.camera_id = c.camera_id
GROUP BY 1;
```

M12, the false-alarm ladder, quantifies the trade-off Question 2.3 demands:

```
WITH j AS (SELECT c.*, v.is_false_alarm, v.confidence
FROM candidates c JOIN verdicts v ON v.candidate_id=c.id)
SELECT 'raw' AS rung, COUNT(*) AS alerts,
COUNT(*) FILTER (WHERE NOT is_false_alarm) AS true_pos FROM j
UNION ALL
SELECT 'confidence>70', COUNT(*), COUNT(*) FILTER (WHERE NOT
is_false_alarm)
FROM j WHERE confidence > 70
UNION ALL
SELECT 'adjudicated', COUNT(*), COUNT(*) FILTER (WHERE NOT
is_false_alarm)
FROM j WHERE NOT is_false_alarm;
```

6.7 — The evidence ledger

```
| Claim | Value | Run ID | Conditions |
|---|---|---|---|
| Motion gate rejection | 91.3% | c1_outage | 6 cams, 4h, STC 01-06, MOG2 default |
| Event vs continuous upload | 1 : 1,340 | c1_outage | 900 kbps source, 47 ev/cam/day |
| Holdout cohort precision drop | 94.1% -> 79.6% | c4_silent | train 01-08, holdout 09-13 |
| Silent fault detection latency | 11 min | c4_silent | completeness < 0.95 for 10m |
```

Footnote your written answers as **[E-c1]**. Character limits are tight, so the compact form matters. **Evaluators reading six essays that all cite the same measurement ledger will notice.**

6.8 — The four postmortems

For each chaos run write: symptom, time to detection, **what you wrongly suspected first**, actual root cause, fix, and the metric that would have caught it earlier.

The “what you wrongly suspected first” line is the one that proves it actually happened. In the founder round, when they hand you a crisis scenario, you will not reason abstractly. You will say “this

looks like C4, where I lost 30 percent of streams and blamed the network before finding the source bitrate change." That is a different category of answer.

Phase 7 — Two HTML Pages

Objective: Something to point a camera at. Nothing more.

Budget: 2 hours. One vanilla HTML file each. No framework, no build step, no component library.

7.1 — Event feed (1 hour)

```
<!doctype html><meta charset=utf-8>
<title>Incidents</title>
<style>
  body{font:15px/1.5 system-
ui;margin:0;padding:20px;background:#fafafa}
  .c{background:#fff;border:1px solid #ddd;border-
radius:6px;padding:12px;margin:8px 0}
  .c.fp{opacity:.45}
  .m{color:#666;font-size:13px}
  .r{margin-top:6px;font-style:italic}
  img{height:80px;margin-right:6px;border-radius:3px}
</style>
<h2>Live incidents</h2>
<div id=f></div>
<script>
async function tick(){
  const r = await fetch('/api/events?limit=40');
  const d = await r.json();
  document.getElementById('f').innerHTML = d.map(e => `
    <div class="c ${e.is_false_alarm ? 'fp' : ''}">
      <b>${e.rule}</b> - ${e.camera_id} / ${e.zone}
      <div class=m>${new Date(e.ts*1000).toLocaleTimeString()}
        - dwell ${e.dwell_s}s - confidence ${e.confidence ?? '-'}
    </div>
    <div>${(e.thumbs||[]).map(t=>``).join('')}</div>
    <div class=r>${e.reasoning ?? 'awaiting adjudication'}</div>
    </div>`).join('');
  }
  tick(); setInterval(tick, 3000);
</script>
```

Show the adjudicator's reasoning text prominently. That sentence is the most differentiated thing in your entire demo. Everyone shows bounding boxes; nobody shows a system explaining why it dismissed an alert.

Greying out false positives rather than hiding them makes the suppression visible, which is the point.

7.2 — Scale simulator (1 hour)

The only custom UI that earns marks, because Grafana cannot do it.

```

<!doctype html><meta charset=utf-8>
<title>Scale model</title>
<body style="font:15px/1.6 system-ui;max-width:640px;margin:40px
auto;padding:0 20px">
<h2>Scale and cost model</h2>
<p style="color:#666">Coefficients measured on this system. See
evidence ledger.</p>
<label>Cameras: <output id=co>1000</output><br>
<input id=c type=range min=100 max=20000 step=100 value=1000
style=width:100%></label>
<label>Retention days: <output id=ro>14</output><br>
<input id=r type=range min=1 max=90 value=14 style=width:100%>
</label>
<table id=t style="width:100%;margin-top:20px;border-
collapse:collapse"></table>
<div id=b style="margin-top:20px;color:#b00"></div>
<script>
const M = {
// <-- REPLACE with your measured
values
  kbps_per_cam: 900,
  events_per_cam_day: 47,
  bytes_per_event: 92000,
  adjud_calls_per_cam_day: 47,
  usd_per_adjud: 0.004,
  cams_per_edge_box: 40,
  usd_per_edge_box_month: 45,
  usd_per_gb_month_storage: 0.015,
  usd_per_gb_egress: 0.02
};
function calc(){
  const n = +c.value, days = +r.value;
  co.textContent = n; ro.textContent = days;
  const ev_gb_mo = n * M.events_per_cam_day * M.bytes_per_event *
30 / 1e9;
  const cont_gb_mo = n * M.kbps_per_cam * 1000 / 8 * 86400 * 30 /
1e9;
  const boxes = Math.ceil(n / M.cams_per_edge_box);
  const rows = [
    ['Edge hardware', boxes * M.usd_per_edge_box_month],
    ['Upstream bandwidth (events)', ev_gb_mo * M.usd_per_gb_egress],
    ['Adjudication', n * M.adjud_calls_per_cam_day * 30 *
M.usd_per_adjud],
    ['Cloud storage', ev_gb_mo * days/30 *
M.usd_per_gb_month_storage]
  ];
  const total = rows.reduce((a,x)=>a+x[1],0);
  t.innerHTML = rows.map(([k,v])=>
    `<tr><td>${k}</td><td align=right>${v.toFixed(0)}</td>
</tr>`).join('')
    + `<tr style="border-top:1px solid #ccc"><td><b>Total /
month</b></td>
    <td align=right><b>${total.toFixed(0)}</b></td></tr>
    <tr><td>Per camera / month</td><td
align=right>${(total/n).toFixed(2)}</td></tr>
    <tr><td>Continuous upload would be</td>
    <td align=right>${cont_gb_mo.toFixed(0)} GB vs
    ${ev_gb_mo.toFixed(1)} GB</td></tr>`;
  const breaks = [];
  if (n > 400) breaks.push('400: manual camera assignment breaks,
need a site coordinator');
  if (n > 3000) breaks.push('3,000: single control plane region
becomes the blast radius');
  if (n > 8000) breaks.push('8,000: single event-store writer
saturates, shard by region');
  if (n > 12000) breaks.push('12,000: move to cell-based
isolation');
  b.innerHTML = breaks.map(x=>'&#9888; '+x).join('<br>');
}
[c,r].forEach(e=>e.oninput=calc); calc();
</script>

```

Replace every value in M with your measured figures. The whole point is that this extrapolates from reality rather than from a spreadsheet you typed.

Dragging the slider past each threshold surfaces a named breaking point with a reason. **This single page simultaneously answers 1.1b, 1.1d, and 1.2**, and no other candidate will have it.

Phase 8 — Demo Recording and Written Answers

Budget: 2 hours.

8.1 — Recording on Fedora (15 min setup)

```
# Wayland (Fedora 36+ default)
sudo dnf install -y wf-recorder
wf-recorder -f demo.mp4 -c libx264 -p preset=veryfast

# or OBS for scene switching
sudo dnf install -y obs-studio
```

Check your session type with `echo $XDG_SESSION_TYPE`. On X11 use `ffmpeg -f x11grab`.

8.2 — Demo beat sheet

Time	Beat	On screen
0:00-0:45	The thesis in one number	“Continuous upload for 5,000 cameras is X Gbps. My measured event stream is Y Mbps.” Show M7
0:45-2:00	Two sites, 12 cameras live	Grafana fleet health, event feed ticking
2:00-4:00	The outage	Toxiproxy cuts the link. Outbox depth climbs. Restore. Ordered backfill. Zero loss proof
4:00-5:30	Bandwidth cap	5 Mbps. Live events flow, backfill throttles
5:30-7:30	The silent fault	C4 replay. Alert fires unprompted. Show detection latency
7:30-9:30	Domain shift	Per-cohort precision. Train vs holdout
9:30-11:00	False alarm ladder	M12 table. Adjudicator reasoning on screen
11:00-13:00	Scale simulator	Drag to 10,000. Breaking points appear
13:00-14:00	What I did not build	Name every stub. Name where you extrapolated

That last beat is counterintuitive and it is where you separate from everyone else. Ending on honest limitations reads as senior. Ending on “and it scales infinitely” reads as junior.

8.3 — Writing the six answers

Character limits: 5,000 / 4,000 / 3,500 / 4,500 / 3,500 / 3,000. Five thousand characters is roughly 800 words. **Density beats coverage.**

For each answer: state the mechanism, cite the measurement, name the trade-off, name what breaks.

Keep these and cut everything else if you run out of room:

- **1.1:** the bandwidth inversion with M7, three named scale transitions with what specifically changes at each, store-and-forward plus manifest reconciliation, and an honest breaking point with a named successor architecture
- **1.2:** measured coefficients, the cascade ratio, three optimisations with their trade-offs
- **1.3:** the completeness metric, the burn-rate alert, your actual C4 detection latency
- **2.1:** the cascade, and the observation that the A-versus-B choice is a false dichotomy because you run the fast path always and the slow path on survivors
- **2.2:** per-cohort confusion matrices, domain shift, registry with shadow and canary as prevention
- **2.3:** the M12 ladder with the precision-recall trade-off quantified, plus the closed policy loop

Appendix A — The Measurement Catalogue

#	Measure	Source	Answers
M1	Bitrate per camera	<code>segment_write.bytes / duration_s</code>	1.2
M2	CPU per recorded stream	cgroup stats, N streams	1.1, 1.2
M3	Motion gate rejection	<code>motion_gate_passed</code> ratio	1.2, 2.1
M4	Detection latency and throughput	<code>detect.latency_ms</code>	2.1
M5	Events per camera per day	<code>candidate</code> count	1.2
M6	Bytes per event	<code>upload.bytes / n_events</code>	1.2
M7	Continuous vs event-only bandwidth	M1 x 86400 vs M5 x M6	1.1, 1.2 headline
M8	Outage recovery curve	<code>outbox.depth</code> over time	1.1c, 1.3
M9	Reconnect behaviour	<code>stream_health.reconnects</code>	1.1c
M10	Precision and recall per rule	<code>verdicts</code> joined to <code>candidates</code>	2.1
M11	Per-cohort accuracy	Group by <code>cameras.cohort</code>	2.2
M12	False alarm ladder	Cumulative filter stages	2.3
M13	Agent fault detection latency	<code>agent_incident.latency_s</code>	1.3
M14	Cameras per edge box	Ramp until p99 write latency degrades	1.1d

Appendix B — Fedora-Specific Gotchas

Issue	Symptom	Fix
SELinux volume mounts	Container cannot read config, starts empty	Append <code>:Z</code> to every <code>-v</code>
Podman rootless ports below 1024	Bind refused	Use high ports, or <code>sysctl net.ipv4.ip_unprivileged_port_start=80</code>
<code>host.containers.internal</code> unresolved	Prometheus cannot scrape	Add <code>--network=host</code> or use the podman0 bridge IP
Stock ffmpeg lacks libx264	Encoding fails	Install RPM Fusion first
<code>pip install</code> into system Python	dnf breaks later	Always use the venv
firewalld blocks podman ports	Works locally, not remotely	<code>sudo firewall-cmd --add-port=3000/tcp --permanent && sudo firewall-cmd --reload</code>
Wayland screen recording fails	Black video	Use <code>wf-recorder</code> , not <code>ffmpeg -f x11grab</code>
<code>tc</code> not found	Command missing	<code>sudo dnf install iproute-tc</code>

Appendix C — Cut Order Under Time Pressure

Cut in exactly this order and stop when you are back on schedule:

1. **Site B** entirely. Lose the Q2.2 cohort, gain 4 hours
2. **Scale simulator**. Lose 4 hours of Q1.2 polish
3. **Fast policy loop**. Keep the adjudicator, drop the automated response
4. **Chaos runs C3 and C4**. Keep C1 and C2
5. **Tripwire rule**. Loitering alone is sufficient
6. **Prometheus and Grafana**. Fall back to `jq` over JSONL

Never cut, under any circumstance: the telemetry schema, the C1 outage run, the adjudicator, or the measurement pass. Those four are the entire submission.

Appendix D — Daily Checklist

Day 1 (8h): Phases 0 and 1 complete, twelve cameras live, `ffprobe` works from the laptop.

Day 2 (8h): Phase 2 complete. **The outage gate passes with zero unsent events.** Do not proceed past this until it does.

Day 3 (7h): Phase 3 complete. Gate rate calibrated to 5-15 percent, candidates flowing.

Day 4 (6h): Phase 4 complete. Verdicts in Postgres with reasoning, `policies` table non-empty.

Day 5 (8h): Phases 5 and 6. Grafana live, all four chaos runs done, evidence ledger populated.

Day 6 (3h): Phases 7 and 8. Two pages, demo recorded, six answers drafted.

Appendix E — The Two Rules That Decide Everything

Write `common/telem.py` in hour one. Not hour twenty. Every measurement is free if the schema exists from the start and unrecoverable if it does not. The failure mode is building the whole system, then trying to extract numbers at the end, and finding you never logged the fields you needed. Then you guess, and your written answers become the same unsupported estimates everyone else submits. That is the difference between data-driven and data-flavoured.

Timebox Oracle Cloud provisioning to two hours. It is the highest-variance item in the plan and the least interesting problem you will solve. Roughly ₹400 for a Hetzner box is not worth a week of retry loops.

Everything else in this document is negotiable.